Available online at www.sciencedirect.com
ScienceDirect
Information Processing Letters 100 (2006) 120–123
Information Processing Letters
www.elsevier.com/locate/ipl

# On the All-Farthest-Segments problem for a planar set of points

Asish Mukhopadhyay *, Samidh Chatterjee, Benjamin Lafreniere

*School of Computer Science, University of Windsor, Windsor, Ontario, Canada*

Received 8 November 2005; received in revised form 23 March 2006; accepted 22 June 2006

Available online 2 August 2006

Communicated by F.Y.L. Chin

**Abstract**

In this note, we outline a very simple algorithm for the following problem: Given a set $S$ of $n$ points $p_1, p_2, p_3, \ldots, p_n$ in the plane, we have $O(n^2)$ segments implicitly defined on pairs of these $n$ points. For each point $p_i$, find a segment from this set of implicitly defined segments that is farthest from $p_i$. The time complexity of our algorithm is in $O(nh + n \log n)$, where $n$ is the number of input points, and $h$ is the number of vertices on the convex hull of $S$.


*Keywords:* Algorithms; Computational geometry; Analysis of algorithms; Design of algorithms; Computational complexity

## 1. Introduction

Geometric optimization is a very active subarea of Computational Geometry. In this paper we study a simple geometric optimization problem. Given a set $S$ of $n$ points $p_1, p_2, p_3, \ldots, p_n$ in the plane, we have $O(n^2)$ segments implicitly defined on pairs of these $n$ points. For each point $p_i$, find a segment from this set of implicitly defined segments that is farthest from $p_i$.

## 2. Previous work

For the *nearest* version of this problem, Daescu and Luo [1] presented an $O(n \log n)$ algorithm; Duffy et al. [4] presented an $O(n^2)$ algorithm for the *all-nearest* version, and also provided evidence that this might be an $O(n^2)$-hard problem. Daescu and Luo [1] and Daescu also presented an $O(n \log n)$ algorithm for the *farthest* version of this problem (see also [2]). Here we show that the *all-farthest* version of the problem can be solved in $O(nh + n \log n)$ time, where $h$ is the number of vertices on the convex hull of the $n$ points.

While it is hard to provide any practical motivation for problems of this type that does not appear contrived, it is intriguing to know whether the *all-farthest* problem can be solved as efficiently as or faster than the *all-nearest* version.

## 3. Characterization of a farthest segment

Let $\overline{p_j p_k}$ be a farthest segment of a point $p_i$. The farthest distance is obtained either by dropping a perpendicular from $p_i$ to the segment $\overline{p_j p_k}$ (Fig. 1) or by joining $p_i$ to the nearer one of the end points $p_j$ and $p_k$ (Fig. 2). We call these two types of farthest segments type $A$ and type $B$, respectively.

* Corresponding author.
*E-mail addresses:* asishm@cs.uwindsor.ca (A. Mukhopadhyay), chatte7@cs.uwindsor.ca (S. Chatterjee), lafreni@uwindsor.ca (B. Lafreniere).

0020-0190/$ – see front matter 
doi:10.1016/j.ipl.2006.06.015

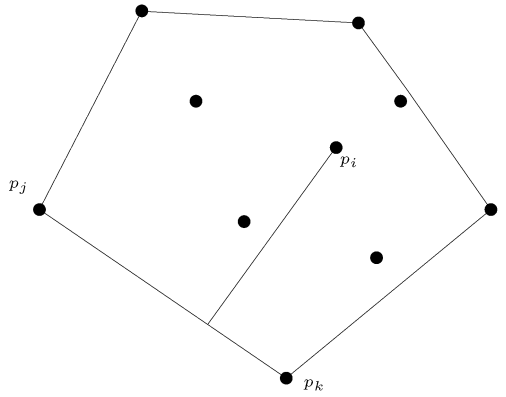


Fig. 1. Farthest distance from $p_i$ to segment $(p_j, p_k)$ is to an intermediate point (type $A$ segment).

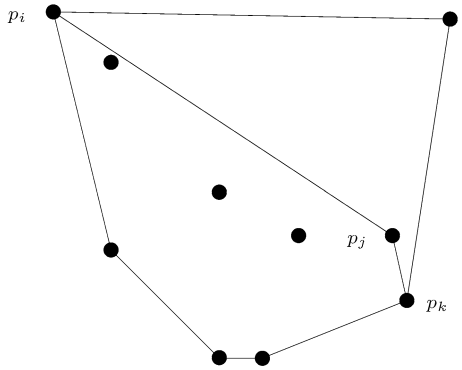


Fig. 2. Farthest distance from $p_i$ to segment $(p_j, p_k)$ is to an endpoint (type $B$ segment).

We design an algorithm by characterizing the two types of segments. To ensure the correctness of the arguments below, we shall assume that no three points of $S$ are collinear.

**Lemma 1.** *If the segment $\overline{p_j p_k}$ is a type A farthest segment for a point $p_i$ then $\overline{p_j p_k}$ is an edge on the convex hull of $S$.*

**Proof.** If the segment $\overline{p_j p_k}$ is not a convex hull edge, then there exists a point $p_l$ of $S$ in the open half-plane defined by the supporting line through $p_j$ and $p_k$ that does not contain $p_i$ (Fig. 3). This gives a segment $\overline{p_j p_l}$ that is farther from $p_i$ than $\overline{p_j p_k}$ since $\overline{p_i p_j}$ is the hypotenuse of the right-triangle formed by $p_i$, $p_j$ and the foot of the perpendicular from $p_i$ to $\overline{p_j p_k}$. This contradicts the assumption that $\overline{p_j p_k}$ is a farthest segment of $p_i$. □

**Lemma 2.** *If the segment $\overline{p_j p_k}$ is a type B farthest segment for a point $p_i$, the farthest distance being the length of $\overline{p_i p_j}$, then either $\overline{p_j p_k}$ is an edge on the convex hull of $S$ or $p_j$ is farthest from $p_i$ among all the points that are interior to the convex hull of the point set, while $p_k$ is a convex hull vertex of the given point set* (*Fig.* 2).

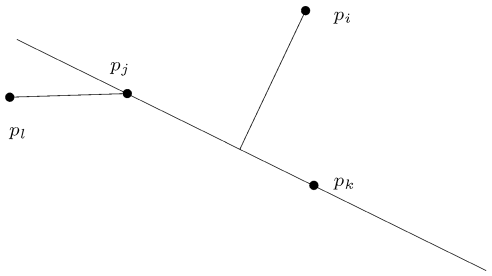


Fig. 3. If a type $A$ segment is not a convex hull edge.

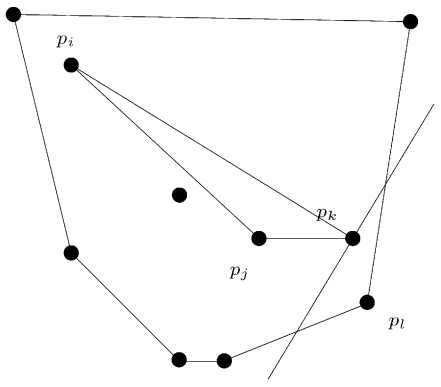


Fig. 4. Illustrating the case when $p_j$ and $p_k$ are both internal vertices of the convex hull of $S$.

**Proof.** Let the farthest distance be realized by joining $p_i$ to $p_j$. Our proof is in three parts, covering the mutually exclusive and exhaustive possibilities that the end points of $\overline{p_j p_k}$ are both points internal to the convex hull of $S$, are both convex hull vertices or one is an internal vertex while the other is a convex hull vertex.

(1) Suppose $p_j$ and $p_k$ are both internal to the convex hull (Fig. 4). If this were true, consider the half-plane defined by a line through $p_k$ orthogonal to $\overline{p_i p_k}$ that does not contain $p_i$. This half plane must contain a vertex $p_l$ of the convex hull of $S$, giving us a segment $\overline{p_k p_l}$ that is farther from $p_i$ than $\overline{p_j p_k}$ and a contradiction. Hence this possibility is excluded.

(2) Suppose $p_j$ and $p_k$ are both vertices of the convex hull. We claim that in this case $\overline{p_j p_k}$ is a convex hull edge. If otherwise, the segment $\overline{p_j p_k}$ divides the convex hull of $S$ into two parts. Consider the convex hull boundary going from $p_j$ to $p_k$ that lies in the part not containing $p_i$ (Fig. 5). Since there is at least one

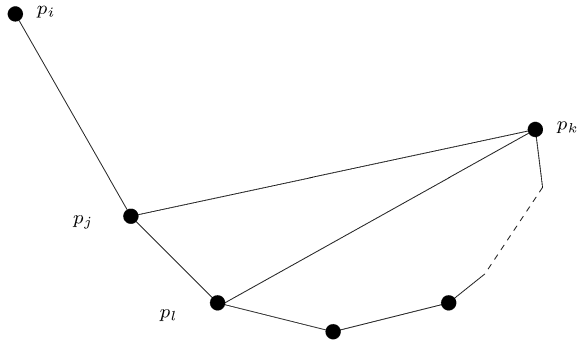


Fig. 5. $p_j$ and $p_k$ are non-adjacent convex hull vertices.

convex hull vertex on this boundary, let $p_l$ be the one closest to $p_j$. Then $\overline{p_l p_k}$ gives us a segment (could be of type $A$ or type $B$) that is farther from $p_i$ than $\overline{p_j p_k}$ as the distances of all points on $\overline{p_l p_k}$ from $p_i$ are greater than the distance from $p_i$ to $p_j$. This proves our claim.

(3) $p_k$ is a convex hull vertex and $p_j$ is an internal vertex. We claim that in this case $p_j$ is farthest from $p_i$ among all internal vertices. Otherwise, let $p_l$ be an internal point that is farther from $p_i$ than $p_j$. There exists a point $p_m$ that lies on the convex hull and is in the halfplane defined by a line through $p_l$ orthogonal to $\overline{p_i p_l}$, not containing $p_i$. This gives us a segment $\overline{p_l p_m}$ farther from $p_i$ than $\overline{p_j p_k}$ and a contradiction.

By (1), (2) and (3), we have proven that the farthest segment from $p_i$ must either be an edge of the convex hull of $S$, or have one end point on the convex hull while the other end point is the farthest from $p_i$ among all internal points. □

With these two lemmas, it is easy to design an efficient algorithm for solving this problem.

## 4. Algorithm

We first construct the convex hull of the point set; then the farthest-point Voronoi diagram of the interior points, if any. The time complexity of each of these two steps is in $O(n \log n)$, [5,6].

For each point $p_i$, we find the farthest segment as outlined in the following algorithm.

**Algorithm** All-Farthest-Segments
**Input**: A set of $n$ points $p_1, p_2, \ldots, p_n$
**Output**: The farthest segment $\overline{p_j p_k}$ for each $p_i$

**for** each $p_i$ **do**

**Step 1**: Find the farthest segment among the edges of the precomputed convex hull; record the segment and the distance.

**Step 2**: Locate $p_i$ in the precomputed farthest point Voronoi diagram of the points interior to the convex hull. Let $p_j$ be its farthest neighbor and record the distance to it from $p_i$. If this distance is smaller than that computed in Step 1 report the segment found in Step 1 and quit, else continue.

**Step 3**: Draw a line orthogonal to the segment $\overline{p_i p_j}$; the other endpoint $p_k$ is a convex hull vertex that lies in the halfplane not containing $p_i$. We find this by a linear search (we can afford this!) on the convex hull boundary. Report $\overline{p_j p_k}$ as the farthest segment.

**od**

## 5. Analysis

The time complexity of Step 1 is in $O(h)$; that of Step 2 is in $O(\log(n-h))$; while that of Step 3 is also in $O(h)$. Putting it all together, the time complexity of All-Farthest-Segments is in $O(nh + n \log n)$.

## 6. Conclusion

It would be interesting to extend this algorithm to finding all $k$th closest segments.

An implementation the above algorithm is available at http://davinci.newcs.uwindsor.ca/~asishm/software.html; use mouse-clicks to create points, and then keep clicking on the *showFarthestSegments* button. For finding out the farthest point we have used a brute-force approach, instead of the farthest-point Voronoi diagram. Hopefully, in a future version of the software we will incorporate this.

There has been some further work on this problem, subsequent to our submission. The details are reported in [3].

## Acknowledgement

We would like to thank the referees for their perceptive comments. The improvements in and clarity of this revised version are due to their comments.

## References

[1] O. Daescu, J. Luo, Proximity problems on line segments spanned by points, in: Proc. of 14th Annual Fall Workshop on Computational Geometry, 2004, pp. 9–10.
[2] O. Daescu, J. Luo, D.M. Mount, Proximity problems on line segments spanned by points, in: Proc. of 17th Canadian Conference on Computational Geometry, 2005, pp. 224–228.
[3] R.L.S. Drysdale, A. Mukhopadhyay, An $o(n \log n)$ algorithm for the all-farthest-segments problem for a planar set of points, Tech-

nical Report 06-011, School of Computer Science, University of Windsor, April 2006.
[4] K. Duffy, C. McAloney, H. Meijer, D. Rappaport, Closest segments, in: Proc. of CCCG 2005, 2005, pp. 229–231.
[5] F.P. Preparata, M.I. Shamos, Computational Geometry: An Introduction, Springer-Verlag, Berlin, 1985.
[6] S. Skyum, A simple algorithm for computing the smallest enclosing circle, Information Processing Letters 37 (1991) 121–125.